Embed Showcase

cyberWriter supports embedding multiple file types inline using the `![[filename]]` vault syntax. This document demonstrates each type.

Data Embeds

CSV Table

CSV files render as styled HTML tables:

Name	Role	Department	Status
Dr. Elena Vasquez	Lead Quantum Architect	Quantum Core	Active
Marcus Kim	Systems Engineer	Infrastructure	Active
Dr. Sarah Chen	Processor Fabrication Lead	Hardware	On Leave
James Wright	Security Analyst	Operations	Active
Aisha Patel	ML Research Lead	AI Division	Active

TSV Table

Tab-separated files work the same way:

Quarter	Revenue	Expenses	Profit
Q1 2026	$2.4M	$1.8M	$0.6M
Q2 2026	$3.1M	$2.0M	$1.1M
Q3 2026	$3.8M	$2.2M	$1.6M
Q4 2026	$4.5M	$2.5M	$2.0M

Media Embeds

Audio

Record a voice note with the mic button, or embed any audio file:

`![[audio/Voice Note 2026-04-13 at 19.54.00.m4a]]`

📝 A Note from JT

00:00 00:00

💡 Try it Click the **mic button** in the editor toolbar to record a voice note. It saves to the `audio/` folder and inserts the embed automatically.

Video

Embed video files up to 50MB inline (larger files show a placeholder):

`![[your-video.mp4]]`

📝 Supported formats mp4, mov, m4v, webm

Images

The classic embed — images render inline:

ℹ️ Image formats png, jpg, jpeg, gif, webp, svg, bmp, heic

Document Embeds

PDF

PDF files render as scrollable inline viewers:

```
![[your-document.pdf]]
```

💡 Try it Drop a `.pdf` file into your vault and embed it with `![[filename.pdf]]`

YouTube

Paste a YouTube link and it renders as a clickable card that opens in your browser:

Rick Astley - Never Gonna Give You Up

▶ YouTube · Click to watch

Drag & Drop

You can also drag files from the vault sidebar directly into the editor:

- **Markdown files** insert as `[[wikilink]]`
- **Everything else** inserts as `![[embed]]` with the full path

Right-click any file in the sidebar → **Copy Wikilink** uses the same syntax.

Text Embeds

Markdown Note

Embed any `.md` file — it renders with full formatting:

> embeds/readme.md
>
> **Voice Note 2026-04-13 at 23.25.51** !audio/Voice Note 2026-04-13 at 23.25.51.m4a**Voice Note 2026-04-13 at 23.26.25** !audio/Voice Note 2026-04-13 at 23.26.25.m4a# Embedded Note This is a **markdown file** embedded inside another document using `!readme.md`. It supports all formatting: - **Bold**, *italic*, `code` - Dr. Elena Vasquez — wikilinks work too - #embedded #demo tags > [!tip] Embeds are recursive > You can embed notes that contain their own formatting, links, and callouts.

Plain Text

Text files render with an accent border:

> embeds/notes.txt
>
> Meeting Notes — Perihelion Sync Date: 2026-03-15 Attendees: Elena, Marcus, Sarah, James Key decisions: - Quantum error correction milestone moved to Q2 - Hardware team needs 3 additional test rigs - Security audit scheduled for April Action items: 1. Elena: Submit revised timeline by Friday 2. Marcus: Provision test environment 3. James: Begin penetration testing prep

RTF

Embed RTF Files right into the doc

> embeds/Why_cyberWriter.rtf
>
> cyberWriter
> The Markdown Editor From the Future
>
> Your Notes Deserve Better Than "Trust Me"
> JT
>
> Chief Analytics Officer | Business Intelligence | AI | Data Engineer | Healthcare Data Strategy | Software Developer
>
> April 10, 2026
>
> Why I built CyberWriter, a native, sandboxed Markdown editor with integrated local AI support - and why the security model of your writing tool actually matter
> I oversee the analytics infrastructure and PHI security policy for one of New York City's largest Federally Qualified Health Center networks -

> 90,000+ patients, clinical data flowing through every system I touch. My day is access controls for internal and external partners, audits and audit logs, HIPAA risk assessments, 42 CFR Part 2 substance use protections, and the constant low-grade dread of knowing that one misconfigured permission can expose someone's HIV status or psychiatric history. Self-hosting PHI is extremely powerful and extremely stressful.
> When I sit down to write - strategic plans, clinical AI deployment analyses, epidemiological/population health assessments - I use a Markdown editor with a locally-hosted LLM. Not a cloud API. A model running on my own hardware, processing my text without it ever leaving my machine. I do this because I've spent two decades learning that the only security guarantee worth anything is a structural constraint. Not a promise. Not a policy. A wall.
> That mindset is what led me to build CyberWriter Not because the world needed another Markdown editor, but because I couldn't find one whose security model I'd trust with the things I actually write - and I write about things that can hurt real people if they leak.
> Here's the problem.
> The plugin problem nobody talks about
> There's a category of popular note-taking and writing apps - you probably already know which ones I'm talking about - built on Electron with rich community plugin ecosystems. They're beautifully designed. They have passionate communities. And their security model is, to put it politely, built on hope.
> Here's how the typical architecture works: the app itself is a relatively thin shell. The real functionality - templates, advanced queries, dataview tables, sync features, collaboration - comes from community-developed plugins. These plugins run as JavaScript in a Node.js context. And in most cases, they have unrestricted access to your filesystem. Not just the folder where your notes live. Your entire filesystem.
> That means a plugin you installed to get a nice calendar view in your sidebar has the same file access as any other process on your machine. Your SSH keys. Your browser cookies. Your tax returns. Your clinical notes. Everything.
> This isn't a theoretical concern. It's how the architecture actually works, and the developers of these apps have publicly acknowledged it. The typical response is some version of: "sandboxing plugins in JavaScript is effectively impossible because there are too many escape hatches - eval, new Function(), child_process, etc."
> That's a fair technical observation. But it's also an indictment of the architectural choice, not a defense of it. If you chose a platform where sandboxing is impossible, and your product handles sensitive personal data, you chose the wrong platform.

> ”Restricted mode” isn’t a real answer
> Some of these apps ship with plugins disabled by default and display a warning before you enable them. That’s better than nothing, but it’s also a fig leaf. The apps are designed to be extended. Their documentation features plugins prominently. Their communities organize around plugin recommendations. Core workflows that users expect - things like advanced templating, structured data queries, or real-time collaboration - simply don’t exist without plugins.
> Telling users ”you can just not use plugins” is like selling a smartphone with no apps and saying ”the phone dialer is very secure.” Technically true. Practically meaningless.
> The workarounds are worse than the problem
> The security-conscious users in these communities have developed workarounds: run the app in a Docker container. Use firejail on Linux. Set up a dedicated VM. Configure Little Snitch to block all network traffic from the app.
> These are real mitigations. They’re also absurd. We’re talking about a note-taking app. If using your writing tool safely requires the same operational security as running a honeypot, something has gone wrong at the architectural level.
> And even the containerized approaches have gaps. One popular Linux packaging format ships with full /home access by default. Users who think they’re sandboxed often aren’t.
> What ”secure by default” actually looks like
> When I started building CyberWriter, I made a deliberate architectural decision: native macOS, Swift/SwiftUI, distributed through the Mac App Store, with full Mermaid and LaTeX suport.
> This isn’t just a technology preference. It’s a security constraint. Here’s what it means in practice:
> App Sandbox enforcement. Every Mac App Store app runs inside Apple’s App Sandbox. CyberWriter literally cannot access files outside its sandbox container unless you explicitly grant access through a system file dialog. This isn’t a setting I chose to enable - it’s enforced by the operating system. A native sandboxed app can’t read your SSH keys, can’t access your browser data, can’t enumerate your home directory. The constraint is structural, not behavioral.
> Code signing and notarization. The binary is signed with my Apple Developer certificate and notarized by Apple before distribution. You can verify exactly who built it and that it hasn’t been tampered with. There’s a real identity - mine - attached to every build.
> No plugin architecture. CyberWriter is batteries-included. There’s no plugin ecosystem, which means there’s no supply chain attack surface. The features you see are the features that exist, and they’re all subject

to the same sandbox constraints as the rest of the app. Nobody's community calendar widget is going to exfiltrate your journal.

No Electron. This isn't about being a platform purist. Electron apps run a full Chromium instance and a Node.js runtime. That's an enormous attack surface for a text editor. Native macOS apps have a fraction of the codebase, a fraction of the dependencies, and a fraction of the vulnerability surface.

Custom Colors for Mermaid Diagrams

Full LaTeX support

A note on open source

I want to be clear: I'm not anti-open-source. I am open source. My most widely used apps - Anubis OSS, an LLM benchmarking tool for Apple Silicon - is fully open-source, and I contribute to other projects. Open source has given me most of the tools I've built my career on. It has 120 stars on Github.

But open source is a development model, not a security model. The ability to inspect code is valuable. It is not, by itself, a guarantee that anyone has inspected it, especially in plugin ecosystems accepting contributions from the general public, where a new update can ship at any time and the previous audit (if there was one) is immediately stale. The open-source projects I trust most are the ones that pair source availability with real architectural constraints on what code can do. Visibility without containment is just transparency about your exposure.

Who should care about this

If you're writing grocery lists and D&D session notes, honestly, use whatever you want. The threat model doesn't justify the tradeoff.

But if you're a clinician writing patient observations, lor drafting a substance use treatment plan that's protected under 42 CFR Part 2 - where any unauthorized disclosure is a federal violation. If you're a lawyer building a case file. If you're a journalist protecting a source. If you're an executive writing strategy documents that would move a stock price. If your writing contains information that could harm real people if it leaks - you should understand the difference between an app that promises not to read your files and an app that can't.

I wouldn't connect an unvetted third-party system to my clinical data infrastructure. Why would I trust one with my own writing about that same data?

What CyberWriter actually is

CyberWriter is an AI-integrated scientific and technical writing tool for macOS. It works with standard Markdown files organized in vault-style folders - your notes stay as plain .md files on your filesystem, no proprietary format, no lock-in. Just drop your .md vault in and keep

> [illegible]
> working
> The AI integration supports both API-based models and locally-hosted LLMs - and this is how I actually use it. My strategic plans, clinical deployment analyses, and internal assessments are written in CyberWriter with a local model running on Apple Silicon. The text I write never leaves my machine. There's no cloud round-trip, no third-party processing, no ambiguity about where my data goes. For someone who spends all day thinking about data governance, that matters.
> This is the tool I trust with my own work. I wouldn't ship something I wouldn't use for the things that actually matter to me.
> It's available on the Apple App Store and directly through LemonSqueezy and it costs less than a month of most cloud-based writing tools.
> I built it because I needed it. I ship it because other people need it too.
> ~John

In Exports

All embedded media is encoded directly into HTML exports, making the output fully portable — share a single `.html` file with audio, video, PDFs, and images included. PDF and DOCX exports render the content inline as well.